

Id_Tópico	Secao	Comando_Conceito	Explicacao	Dica_Pratica
1	1. Consultas de Dados	SELECT	Seleciona as colunas específicas que queres ver.	Usa SELECT * para ver todas as colunas.
2	1. Consultas de Dados	FROM	Indica a tabela específica de onde os dados vão ser retirados.	Define a base principal da consulta.
3	1. Consultas de Dados	LIMIT	Limita os resultados a um número específico de linhas.	Útil para testar rapidamente ou ir buscar Top N.
4	1. Consultas de Dados	OFFSET	Ignora as primeiras X linhas antes de começar a devolver resultados.	Usado em paginação junto com o LIMIT.
5	1. Consultas de Dados	ORDER BY	Ordena os resultados por uma ou mais colunas (ASC ou DESC).	Cuidado para o nome da coluna no ORDER BY coincidir com o do SELECT. Podes ordenar por condições (ex: Tipologia IS NULL).
6	1. Consultas de Dados	DISTINCT	Retorna apenas valores únicos, removendo duplicados da lista.	Cuidado ao misturar com SUM, pois pode apagar valores idênticos.
7	1. Consultas de Dados	AS	Cria um apelido (alias) temporário para uma coluna ou tabela.	Facilita drasticamente a leitura e escrita do código.
8	1. Consultas de Dados	IN	Verifica se um valor corresponde a qualquer valor dentro de uma lista.	Exemplo: WHERE idade IN (18, 20, 25).
9	2. Filtros e Condições	WHERE	Filtra os dados com base numa condição específica antes do agrupamento.	Aplica-se a linhas individuais antes do GROUP BY.
10	2. Filtros e Condições	Operadores Lógicos (AND, OR, NOT)	AND exige tudo verdadeiro; OR basta uma verdadeira; NOT nega.	Usa parênteses () para forçar a prioridade lógica.
11	2. Filtros e Condições	Operadores de Comparação (=, !=, <>)	Compara valores usando = (igual) ou != / <> (diferente/não igual).	Essencial para filtrar estados que não queremos (ex: Estado != 'Ativo').
12	2. Filtros e Condições	IS NULL / IS NOT NULL	Filtra linhas onde o campo está vazio (nulo) ou preenchido com dados.	IS NULL apanha os "Zezitos" sem registo; IS NOT NULL exige que o campo exista.
13	2. Filtros e Condições	BETWEEN ... AND ...	Filtra valores que estão dentro de um intervalo fechado.	Muito usado para datas e números.
14	2. Filtros e Condições	LIKE (% e _)	Procura por um padrão específico numa coluna de texto.	% apanha qualquer texto; _ apanha um caractere exato.
15	3. Manipulação de Texto	IFNULL()	Verifica se um campo é NULL e substitui-o por um valor plano B.	Exemplo: IFNULL(Tipologia, 'Sem Cartão'). Evita espaços vazios feios.
16	3. Manipulação de Texto	UPPER() / LOWER()	Converte todo o texto de uma coluna para maiúsculas ou minúsculas.	Excelente para normalizar dados mal introduzidos na base de dados.
17	4. Agregação	COUNT()	Conta o número total de linhas ou registos.	Nota: Ignora valores NULL a menos que uses COUNT(*).
18	4. Agregação	SUM()	Soma os valores numéricos de uma coluna.	Cuidado com duplicações geradas por múltiplos JOINS mal estruturados.
19	4. Agregação	AVG() / MIN() / MAX()	Calcula a média, o valor mais baixo e o mais alto de uma coluna.	Funções matemáticas clássicas de suporte à gestão.
20	4. Agregação	ROUND()	Arredonda números decimais para uma leitura financeira limpa.	Exemplo: ROUND(Valor, 2).
21	4. Agregação	GROUP BY	Agrupa linhas que têm os mesmos valores.	Obrigatório sempre que usas funções como COUNT ou SUM junto com colunas normais.
22	4. Agregação	HAVING	Funciona como um WHERE, mas filtra os dados APÓS o agrupamento.	Usado para filtrar resultados de agregações (ex: HAVING Total > 100).
23	5. Relacionamentos	INNER JOIN (JOIN)	Junta registos de duas tabelas apenas quando há correspondência exata.	O que não tem par é descartado de imediato.
24	5. Relacionamentos	LEFT JOIN	Retorna tudo da tabela da esquerda e os correspondentes da direita.	Se não houver par, preenche com NULL (salva os Zezitos).
25	5. Relacionamentos	RIGHT JOIN	Faz o oposto do Left Join, priorizando a tabela da direita.	Dependendo do motor SQL, pode comportar-se de formas específicas.

26	5. Relacionamentos	ON	Define a chave de ligação obrigatória entre as tabelas num Join.	Exemplo: <code>ON Clientes.Id = Cartoes.Id_Cliente.</code>
27	5. Relacionamentos	Filtros: ON vs WHERE	Colocar no ON atua como porteiro preventivo; no WHERE corta o resultado final.	Colocar o Estado = 'Ativo' no ON transforma inativos em NULL via Left Join!
28	6. Subconsultas	Subquery (Subconsultas)	Executa uma mini-consulta isolada dentro de outra estrutura.	Evita o efeito multiplicador indesejado em tabelas com relação 1 para muitos.
29	7. DML	INSERT INTO	Insere uma nova linha de dados na tabela.	Estrutura: <code>INSERT INTO tabela VALUES (...).</code>
30	7. DML	UPDATE ... SET	Atualiza dados já existentes numa tabela.	⚠ Atenção pedagógica: Usa sempre com WHERE para não alterar a tabela toda!
31	7. DML	DELETE FROM	Apaga linhas específicas de uma tabela.	⚠ Atenção: Sem WHERE, apagas todos os dados guardados!
32	8. DDL	CREATE TABLE	Cria uma tabela nova do zero definindo colunas, PK e FK.	A base de qualquer projeto de engenharia de dados.
33	8. DDL	ALTER TABLE	Altera a estrutura de uma tabela já existente.	Permite adicionar ou apagar colunas dinamicamente.
34	8. DDL	DROP TABLE	Elimina permanentemente a tabela inteira e todos os dados.	Ações irreversíveis – cuidado no ambiente de produção!
35	8. DDL	DEFAULT	Define um valor padrão automático para uma coluna.	Preenche automaticamente caso o utilizador nada insira.
36	8. DDL	Tipos de Dados (TEXT, INTEGER, REAL)	Define a natureza dos dados que a coluna vai guardar (texto, números inteiros ou decimais).	Usa INTEGER para IDs, TEXT para nomes e REAL para valores financeiros.
37	8. DDL	PRIMARY KEY	Identifica de forma única cada registo de uma tabela e impede valores duplicados.	Geralmente combinado com AUTOINCREMENT para gerar IDs automáticos.
38	8. DDL	FOREIGN KEY	Chave estrangeira usada para relacionar duas tabelas com base na Primary Key de outra.	Garante a integridade referencial dos dados do projeto.
39	8. DDL	NOT NULL / UNIQUE	Restrições que obrigam o campo a ter sempre preenchimento obrigatório (NOT NULL) ou valores únicos (UNIQUE).	Essencial para evitar emails duplicados ou campos de nome vazios.
40	8. DDL	CREATE VIEW / DROP VIEW	Cria ou elimina uma tabela virtual baseada numa query guardada.	Excelente para simplificar relatórios complexos sem duplicar dados no disco.
41	2. Filtros e Condições	CASE WHEN (Lógica Condicional)	Permite criar colunas dinâmicas baseadas em condições, funcionando como um IF/ELSE no SQL.	Exemplo: <code>CASE WHEN Valor > 100 THEN 'Alto' ELSE 'Baixo' END.</code>
42	5. Relacionamentos	UNION / UNION ALL	Junta os resultados de duas consultas diferentes numa única listagem vertical.	O UNION remove duplicados, enquanto o UNION ALL mantém tudo (sendo mais rápido).
43	6. Subconsultas	EXISTS / NOT EXISTS	Verifica se existem registos numa subconsulta sem necessidade de fazer JOINS pesados.	Muito útil para validar se um cliente tem faturas ativas de forma eficiente.
44	3. Manipulação de Texto/Data	STRFTIME() / DATETIME()	Funções do SQLite para formatar, extrair ou calcular datas e horas.	Indispensável para filtrar relatórios de transporte por mês, ano ou dia específico.